# Study Guide: Scaling Agile Methods and Organizational Considerations

This study guide provides a comprehensive review of the complexities involved in scaling agile methods for large-scale software development and the challenges of implementing these practices within large organizations.

## Short-Answer Quiz

1. **Why is a completely incremental approach to requirements engineering considered impossible for large-scale systems?**
2. **What defines a "brownfield system" in the context of software development?**
3. **How do external rules and regulations impact the development process of large systems?**
4. **In IBM's Agility at Scale model, what distinguishes "Disciplined Agile Delivery" from core agile practices?**
5. **What is the function of a "Scrum of Scrums" in multi-team agile projects?**
6. **Why might project managers in large organizations be reluctant to adopt agile methods?**
7. **How does the concept of refactoring conflict with traditional change management procedures?**
8. **What is the role of "product architects" in a multi-team Scrum environment?**
9. **Why is it difficult to maintain a single product owner or customer representative for large systems?**
10. **What is "release alignment," and why is it necessary for large-scale agile development?**

---

## Answer Key

1. **Answer:** Large systems require some initial work on software requirements to identify different system parts for different teams and to satisfy contractual obligations. While details can be developed incrementally, an up-front overview is essential for defining the work breakdown and system architecture.
2. **Answer:** Brownfield systems are those that must include and interact with a number of existing legacy systems. Development is often constrained because requirements are centered on these interactions, and changes to existing systems require complex negotiations with other system managers.

3. **Answer:** Large systems are often constrained by external regulations that mandate specific types of system documentation or compliance requirements. These rules limit development flexibility and may require developers to complete extensive process documentation that is not typically part of core agile methods.
4. **Answer:** Disciplined Agile Delivery involves adapting core practices to a disciplined organizational setting by recognizing that teams cannot focus solely on construction. It requires incorporating other software engineering stages, such as requirements engineering and architectural design.
5. **Answer:** A Scrum of Scrums is a daily meeting where representatives from various teams meet to discuss their progress, identify inter-team problems, and plan the day's work. These meetings are often staggered to allow representatives to attend multiple sessions if necessary.
6. **Answer:** Project managers without agile experience may view the approach as a high-risk unknown that could negatively affect their project outcomes. They may prefer traditional, predictable methods over the informal decision-making and lack of documentation often associated with agile.
7. **Answer:** Traditional change management requires all system changes to be approved in advance to control costs and predict impacts. Refactoring, a core agile practice, allows any developer to improve code without external approval, creating a direct conflict with bureaucratic control processes.
8. **Answer:** In multi-team environments, each team chooses a product architect. These individuals collaborate with their counterparts from other teams to design, communicate, and evolve the overall system architecture across the entire project.
9. **Answer:** Because large systems involve a diverse set of stakeholders with different perspectives (such as administrators, end-users, and managers), it is impossible for one person to represent all interests. Instead, different representatives must be involved for different parts of the system and negotiate throughout the process.
10. **Answer:** Release alignment is the practice of synchronizing the dates of product releases across multiple development teams. This ensures that when increments are finished, they can be integrated into a complete, demonstrable system.

---

# Essay Questions

1. **The Pragmatic Choice:** Discuss the author's argument that the label of "plan-driven" or "agile" is less important than meeting the needs of the buyer. How should developers choose their methods based on system type, team capability, and organizational culture?
2. **The Complexity of Large Systems:** Analyze the six principal factors (such as "System of Systems" and "Diverse Stakeholders") that contribute to the complexity of large-scale software. Explain how these factors specifically hinder the application of "pure" agile methods.

3. **Scaling Models:** Compare the Scaled Agile Framework (Leffingwell) with IBM's Agility at Scale Model. Focus on how these models transition from core agile development to handling factors like geographic distribution and technical complexity.
4. **Cultural Barriers to Agility:** Evaluate the difficulties of "scaling out" agile methods across a large organization with established bureaucratic procedures. What role do "evangelists" and pilot projects play in overcoming cultural resistance?
5. **Documentation and Communication:** Explain why large-scale agile development requires more up-front design and formal documentation than small-scale agile projects. Detail the specific types of documentation and communication channels (e.g., wikis, social networking, videoconferences) necessary for success.

---

# Glossary of Key Terms

| Term | Definition |
|---|---|
| **Agility at Scale** | The final stage of the IBM scaling model which accounts for complexity such as distributed development, legacy environments, and regulatory compliance. |
| **Brownfield Systems** | Systems that are developed to include or interact with existing, established software systems. |
| **Change Management** | The process of controlling changes to a system to ensure costs are managed and impacts are predictable; often requires advance approval for changes. |
| **Continuous Integration** | An agile practice where the system is built every time a developer checks in a change; often difficult to achieve in large systems with separate programs. |
| **Disciplined Agile Delivery** | An evolution of agile where core practices are adapted to include requirements and architectural design within an organizational setting. |

| | |
|---|---|
| **Plan-driven Development** | An approach to software engineering where requirements and design are extensively documented before implementation begins. |
| **Product Owner** | A role responsible for representing the customer's interests; in large projects, multiple product owners may be used. |
| **Refactoring** | The process of improving code structure and readability without changing its external behavior; a core practice of agile development. |
| **Scrum of Scrums** | A meeting of representatives from multiple Scrum teams to coordinate work and resolve cross-team issues. |
| **System of Systems** | A large-scale system composed of several separate, communicating systems, often developed by different teams in different locations. |